LAB ASSIGNMENT

NLP Foundations: Handwriting OCR Evaluation

Deepening your understanding after Lab 6

Course	**Artificial Intelligence**
Practical	Lab 6 of 8 - Handwriting OCR Evaluation
Student details	Name: Tengku Ameera Nadhirah Binti Tengku Azmi Student ID: 24005574

Part A - Understand the concepts

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

NLP Pipeline Stages	Description	Example
Acquire	Obtaining text or raw data from image, audio or keyboard input	The handwritten medicine-name image is loaded and read at this stage
Normalise	Cleaning the data by converting it into a consistent form	The text acquired from the images will be converted into lowercase, all special character or space is removed and only letters and digits are kept.
Represent	Convert text into a representation that computers can process	Extract the text region or line in the image word counts, n-grams and vectors where the result will be represented as a character string. Also included image preprocessing to make the writing in the images more visible.
Model	Apply the NLP algorithm to perform the task of prediction, searching or classification	Tesseract OCR with –oem 1 and --psm 7 to reads the image and outputs a string such as “Beecky”
Evaluate	Compare the system’s output with the correct reference text	Compare normalized prediction to normalized label using exact match accuracy, mean normalized edit distance, CER and blank-prediction count

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Segmentation finds the text region or line in the image and splits it into words or characters that can be read separately. Failure example based on Lab 6 baseline is 42 raw images returned empty string which means the engine could not locate any text from the handwritten images even after preprocessing.

Character recognition decides what character or words that came up. It maps out the shape to a letter or digit. Failure example can be the engine returned "nidu" for the reference "nims", it can read the lines but the cursive m and s were read as other letters.

It is useful to identify which stage caused an error as the fix depends on the cause. A segmentation failure points to image-level remedies while character recognition failure points to the recognizer itself.

Q3. Text normalisation

The reference is "Tab Azimax 40mg" and the OCR output is "tab azimax 40 mg". Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

Reference: Tab Azimax 40mg → lowercase "tab azimax 40g" → remove spaces "tabazimax40g"

OCR Output: already lowercase "tab azimax 40g" → remove spaces "tabazimax40g"

Both normalized strings counts as an exact match even though raw strings differ. The same rule applied to both because the comparison should measure recognition errors only and to ensure consistency throughout the whole system so that the metrics is fair and comparable.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

Spaces might separate meaningful items. For example, "Tab Azimax 40g" and "Tab Azimax 4.0g" might be treated the same way after normalisation as it will both be "tabazimax40g" which will be wrong. The normalisation rule can be changed for this situation where spaces can be kept for more clarity in doses.

Part B — Work with OCR measurements

Q5. Levenshtein distance

Calculate the Levenshtein distance between “breeky” (reference) and “beeky” (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

	//	b	e	e	k	y
//	0	1	2	3	4	5
b	1	0	1	2	3	4
r	2	1	1	2	3	4
e	3	2	1	1	2	3
e	4	3	2	1	2	3
k	5	4	3	2	1	2
y	6	5	4	3	2	1

The edit operation only came down to 1 character deletion in the reference which equals to 1 insertion in the prediction so the Levenshtein distance = 1

Normalised Edit Distance = 1/6 = 0.167

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

CER = (4+2+1) / 35 = 7/35 = 0.2

Meaning: On average, 20% of the reference characters are in error. The lower CER is, the better is the OCR performance.

Yes, CER can be greater than 1. This might happen when the total number of edit exceeds the number of reference characters.

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Exact-match accuracy: Percentage of images where the normalized OCR predictions exactly matched the normalized reference. Higher exact-match accuracy is better.

Mean Normalised Edit Distance: Average edit distance divided by the length of the longer string for each image. Lower normalized edit distance is better.

CER: Total character edits divided by total reference characters. Lower CER is better.

Blank-predictions: Number of images where the normalized OCR output is empty. Fewer blank predictions are better.

Exact-match accuracy alone is sufficient as it provide the objective, all-or-nothing result of the comparisons between OCR predictions and references.

Part C — Interpret the Lab 6 experiment

Use the results from Lab 6. Both pipelines were evaluated on the same 89 labelled images, with the same normalisation rule and fixed Tesseract configuration.

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

1. The preprocessed results shows fewer blank predictions of 15 compared to the baseline which produce 42 blank predictions.
2. The preprocessed results shows lower character error as the CER is 0.80 compared to the 0.88 CER of the baseline.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Exact match needs every character of the name to be right, but preprocessing only helps the engine detect and segment the text better. It improved text recovery and reduced blanak predictions but character-recognition errors still remained as Tesseract is trained on printed fonts and cannot reliably recognise cursive hand-writing.

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

It makes the experiment more controlled and easier to conclude. If several settings was changed, we will not be able to tell which changes caused an improvement or a drop. This will cause the conclusions of preprocessing become unreliable.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;
Runtime failure. This might be caused by the engine not being able to recognize the text in the image and will be counted as failed recognition.

(b) OCR returns an empty string;
No text detected. This might happened when segmentation process failed and no text was detected or recognized.

(c) OCR returns “nidu” when the reference is “nims;
Character confusion which happened when characters was misread due to recognition failure.

Part D — Apply your understanding

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

1. Use a handwriting-specific recognizer: It targets the recognition error which preprocessing can help but cannot fix. An experiment can be conducted where the same hand-writing images was processed in the recognizer and the Tesseract. The result can be evaluated with the same metrics and compared whether it helps.
2. Contrast enhancement: It targets the segmentation error where no text was detected due to poor image preprocessing. The same test can be run but with added step of contrast enhancement to the current preprocessing to see if the metrics improve or not.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

- Wrong drug identification, similar looking name of drug could be dispensed.
- Dosage or unit error, misread digits or unit can cause overdose or underdose to patients.
- Missing prescriptions, blank output may cause a prescription to be missing .

A safer workflow would include running a handwriting-specific OCR that outputs text with confidence score and match it against approved drug and dose database. Human checking should still be a priority as the OCR only assist the human and never should decide the medication on its own.

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

Preprocessing might improve blank predictions but it is still not exact-match accurate as long as we use the wrong OCR for the wrong raw data processing.

One concept I found difficult would be the normalized edit distance vs CER as it looked the same but it was actually averaged differently. For example, “breeky” and “beeky” is one deletion different so distance is 1,but normalized is calculated as 1/6 = 0.167 which means it contributes 1 edit over 6 reference characters to the CER.